

Ipopt revisited: Strategies to solve edge cases

Horacio Lisdero Scaffino

May 21, 2026

Agenda

- A tour of Ipopt

 - History

 - Techniques applied

- An attempt of a simple IPM algorithm

 - Challenges encountered

 - Results

- A tour of ripopt

- Experiments

 - What can we learn

- Conclusion

Agenda

A tour of Ipopt

History

Techniques applied

An attempt of a simple IPM algorithm

Challenges encountered

Results

A tour of ripopt

Experiments

What can we learn

Conclusion

Ipopt: Interior Point OPTimizer

Popular open-source non-linear optimization package

Bindings available for:

- ▶ AMPL (algebraic modeling language)
- ▶ Fortran
- ▶ GAMS (general algebraic modeling system)
- ▶ Julia
- ▶ Matlab / Scilab
- ▶ .NET languages: C#, F# and Visual Basic.NET
- ▶ Python (several wrappers available)
- ▶ R
- ▶ Rust

Ipopt: The origins

- ▶ PhD thesis by Andreas Wächter in 2002 at Carnegie Mellon [1]
- ▶ Since 2002 part of the COIN-OP foundation [2]
- ▶ Original implementation in Fortran 77 (1.x and 2.x), now deprecated
- ▶ Open source since the start
- ▶ Benchmarked against KNITRO (1999) and LOQO (1999) [3, 4]

Ipopt: Subsequent paper

- ▶ Paper by Wächter and Biegler in 2006 [5]
- ▶ More focused on the central algorithm of Ipopt
- ▶ Features described more in detail
 - ▶ Feasibility restoration phase,
 - ▶ Inertia correction,
 - ▶ Heuristics...
- ▶ More benchmarks: CUTEr test set [6]

Ipopt: C++ version

- ▶ IBM Research decided to invest in an open source re-write of Ipopt in C++
- ▶ With the help of Carl Laird, the code was re-implemented from scratch [7]
- ▶ 3.x published in Aug 2005
- ▶ It is still actively developed on GitHub

Summary

- ▶ Primal-dual interior-point method with filter line-search globalization
- ▶ Sparse symmetric KKT solves with inertia monitoring/correction
- ▶ Inertia correction and regularization to obtain the proper search direction
- ▶ Feasibility restoration phase for difficult or infeasible iterates
- ▶ Second-order correction steps to reduce Maratos-type effects
- ▶ Practical heuristics for robustness and faster performance

Interior Point Methods

Agenda

A tour of Ipopt

History

Techniques applied

An attempt of a simple IPM algorithm

Challenges encountered

Results

A tour of ripopt

Experiments

What can we learn

Conclusion

Introduction

I wanted to play around with code and not just present equations
and thus produce some nice visualizations :)

Introduction

I wanted to play around with code and not just present equations

and thus produce some nice visualizations :)

I wanted to see if optimization in pure Rust is possible

memory safety, performance, modern language for low-level applications, ...

Introduction

I wanted to play around with code and not just present equations
and thus produce some nice visualizations :)

I wanted to see if optimization in pure Rust is possible
memory safety, performance, modern language for low-level applications, ...

I wanted to contribute something back to the community,
i.e, **not** throw away the Python code 1 week after the seminar

The original plan

1. Select the most popular optimization package
2. Implement a simple interior-point method
3. Compare it with Ipopt
4. Profit!



argmin

argmin v0.11.0

Numerical optimization in pure Rust

[Homepage](#) [Documentation](#) [Repository](#)

📄 All-Time: 3,049,452

📄 Recent: 531,942

🔄 Updated: 7 months ago

Website [8]

Pure Rust 🦀

We aim at providing a wide range of solvers implemented in pure Rust.

Consistent interface 🐦

Easily plug your optimization problem in different solvers thanks to a consistent interface.

Type agnostic 🔧

Represent your variables, gradients, Jacobians and Hessians using your own types.

Various math backends 🇺🇸

Use Vecs, ndarray, or nalgebra for behind-the-scenes math. Or implement your own backend.

Observers 🗒️

Observe the progress of your optimization by logging to screen or file or implement your own observer.

Checkpointing 🚩

Mitigate the negative effects of crashes in unstable computing environments by saving regular checkpoints.

Extensible 🦋

Most features can be exchanged for user supplied implementations thanks to Rusts powerful generics and traits.

Development framework 🛠️

Use argmins tools and ecosystem to implement your own solver - with all the features mentioned above.

discarded several libraries: dealt only with LP (good_lp), used novel algorithms (clarabel), or were too small (interiors)

argmin: Code example

```
1  impl<O, P, G, H, F> Solver<O, IterState<P, G, (), H, (), F>> for Newton<F>
2  where
3      O: Gradient<Param = P, Gradient = G> + Hessian<Param = P, Hessian = H>,
4      P: Clone + ArgminScaledSub<P, F, P>,
5      H: ArgminInv<H> + ArgminDot<G, P>,
6      F: ArgminFloat,
7  {
8      fn next_iter(
9          &mut self,
10         problem: &mut Problem<O>,
11         mut state: IterState<P, G, (), H, (), F>,
12     ) -> Result<(IterState<P, G, (), H, (), F>, Option<KV>), Error> {
13         let param = state.take_param().ok_or_else(argmin_error_closure!(
14             NotInitialized,
15             concat!(
16                 "`Newton` requires an initial parameter vector. ",
17                 "Please provide an initial guess via `Executor`s `configure` method."
18             ))
19         );
20         let grad = problem.gradient(&param)?;
21         let hessian = problem.hessian(&param)?;
22         let new_param = param.scaled_sub(&self.gamma, &hessian.inv()?.dot(&grad));
23         Ok((state.param(new_param), None))
24     }
25 }
```


Math backends

The math backend is implemented in a separate crate
`argmin-math`

It is trait-based: interfaces define the methods required from the
linear algebra backend

- ▶ `nalgebra` [9]
- ▶ `faer` [10]

Add a solver

Add a trait:

```
1  pub trait ArgminLuSolve<T> {  
2      /// Solve linear system using LU decomposition.  
3      fn lu_solve(&self, rhs: &T) -> Result<T, Error>;  
4  }
```

And the corresponding implementation:

```
1  impl<N, D, C, S> ArgminLuSolve<OMatrix<N, D, C>> for SquareMatrix<N, D, S>  
2  where  
3      N: ComplexField,  
4      D: Dim + DimMin<D, Output = D>,  
5      C: Dim,  
6      S: Storage<N, D, D>,  
7      DefaultAllocator: Allocator<N, D, D> + Allocator<N, D, C> + Allocator<N, D>,  
8  {  
9      #[inline]  
10     fn lu_solve(&self, rhs: &OMatrix<N, D, C>) -> Result<OMatrix<N, D, C>, Error> {  
11         let lu = LU::new(self.clone_owned());  
12         lu.solve(rhs).ok_or_else(|| LuSolveError {}.into())  
13     }  
14 }
```

No constraints?

```
1  /// Defines equality constraints for a problem.
2  ///
3  /// It follows the form  $h(x) = (0, \dots, 0)$ , where  $h$  is a vector field.
4  pub trait EqualityConstraint {
5      /// Type of the parameter vector
6      type Param;
7      /// Type of the return value of the equality constraints
8      type Output;
9
10     /// Compute all equality constraints
11     fn equality_constraint(&self, param: &Self::Param) -> Result<Self::Output, Error>;
12 }
```

And their derivatives...

```
1  /// Defines the Jacobian of the vector field representing the equality constraints for a  
2  ↪ problem.  
3  pub trait EqualityConstraintJacobian {  
4      /// Type of the parameter vector  
5      type Param;  
6      /// Type of the Jacobian  
7      type Jacobian;  
8  
9      /// Compute the Jacobian of the equality constraints  
10     fn equality_constraint_jacobian(&self, param: &Self::Param) -> Result<Self::Jacobian,  
11         ↪ Error>;  
12     bulk!(equality_constraint_jacobian, Self::Param, Self::Jacobian);  
13 }
```

And the Hessian of the Lagrangian...

```
1  pub trait LagrangianHessian {
2      /// Type of the parameter vector
3      type Param;
4      /// Type of the multipliers associated with equality constraints
5      type MultipliersEq;
6      /// Type of the multipliers associated with inequality constraints
7      type MultipliersIneq;
8      /// Type of the Hessian
9      type Hessian;
10
11     /// Compute the Hessian of the Lagrangian
12     fn lagrangian_hessian(
13         &self,
14         param: &Self::Param,
15         multipliers_eq: &Self::MultipliersEq,
16         multipliers_ineq: &Self::MultipliersIneq,
17     ) -> Result<Self::Hessian, Error>;
18
19     bulk!(
20         lagrangian_hessian,
21         Self::Param,
22         Self::MultipliersEq,
23         Self::MultipliersIneq,
24         Self::Hessian
25     );
26 }
```

And iteration state for NLP...

- ▶ param: Option<P>
- ▶ slacks: Option<S>
- ▶ best_param: Option<P>
- ▶ best_slacks: Option<S>
- ▶ cost: F
- ▶ best_cost: F
- ▶ target_cost: F
- ▶ grad: Option<G>
- ▶ lagrangian_hessian: Option<LH>
- ▶ equality_constraints: Option<EqC>
- ▶ inequality_constraints: Option<IneqC>
- ▶ equality_constraint_jacobian: Option<EqCJ>
- ▶ inequality_constraint_jacobian: Option<IneqCJ>
- ▶ mu: Option<F>
- ▶ lambda_eq: Option<LambdaEq>
- ▶ lambda_ineq: Option<LambdaIneq>
- ▶ alpha_primal: Option<F>
- ▶ alpha_dual: Option<F>
- ▶ inf_pr: Option<F>
- ▶ inf_du: Option<F>
- ▶ compl_inf: Option<F>
- ▶ iter: u64
- ▶ last_best_iter: u64
- ▶ max_iters: u64
- ▶ counts: HashMap<String, u64>
- ▶ counting_enabled: bool
- ▶ time: Option<Duration>
- ▶ termination_status: TerminationStatus

Finally! IPM

`nalgebra_backend.rs`: All the linear-algebra-specific logic for `nalgebra`, the only supported backend

`linsearch.rs`: The custom backtracking linear search. The existing implementation could not be reused

`iteration.rs`: A single iteration of the interior-point method

`interiorpointmethod.rs`: The solver. The high-level management of the IPM algorithm.

Agenda

A tour of Ipopt

History

Techniques applied

An attempt of a simple IPM algorithm

Challenges encountered

Results

A tour of ripopt

Experiments

What can we learn

Conclusion

riopt: A Rust re-write from February 2026

Coded in 60/70 hs during the course of 6 weeks heavily relying on Claude Code [11].

According to the benchmarks CUTEst and xxxx, it already beats lpopt.

Interesting insights into licensing issues and thought process documented in the repo.

Agenda

A tour of Ipopt

- History

- Techniques applied

An attempt of a simple IPM algorithm

- Challenges encountered

- Results

A tour of ripopt

Experiments

- What can we learn

Conclusion

Agenda

A tour of Ipopt

- History

- Techniques applied

An attempt of a simple IPM algorithm

- Challenges encountered

- Results

A tour of ripopt

Experiments

- What can we learn

Conclusion

Conclusion

Ipopt's performance can still be improved.

Different methods complement each other: By combining them, edge cases can be solved more efficiently.

Implementation of a state-of-the-art optimization package may be in the reach of modern AI agentic coding tools.

Questions?

Links

Presentation:

https://github.com/hlisdero/JMCS_Seminar_Applied_Optimization

Modified argmin: <https://github.com/hlisdero/argmin>

riopt: <https://github.com/jkitchin/riopt>

ipopt-rs: <https://github.com/elrnv/ipopt-rs>

Bibliography



A. Wachter, *An interior point algorithm for large-scale nonlinear optimization with applications in process engineering*.

Carnegie Mellon University, 2002.



C.-O. Foundation, “Coin-op ipopt.” <https://github.com/coin-or/Ipopt>.

Accessed: 2026-04-15.



R. H. Byrd, M. E. Hribar, and J. Nocedal, “An interior point algorithm for large-scale nonlinear programming,” *SIAM Journal on Optimization*, vol. 9, no. 4, pp. 877–900, 1999.



R. J. Vanderbei, “Loqo: An interior point code for quadratic programming,” *Optimization methods and software*, vol. 11, no. 1-4, pp. 451–484, 1999.



A. Wächter and L. T. Biegler, “On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming,” *Mathematical programming*, vol. 106, no. 1, pp. 25–57, 2006.



N. I. Gould, D. Orban, and P. L. Toint, “Cuter and sifdec: A constrained and unconstrained testing environment, revisited,” *ACM Transactions on Mathematical Software (TOMS)*, vol. 29, no. 4, pp. 373–394, 2003.



COIN-OR Foundation, *Ipopt: Interior Point OPTimizer*, 2026.



S. Kroboth, “argmin: Mathematical optimization in pure rust.” <https://argmin-rs.org/>.

Accessed: 2026-03-28.



S. Crozet, “nalgebra: Linear algebra library for rust.” <https://nalgebra.rs/>.

Accessed: 2026-03-28.



S. Quiñones, “faer: linear algebra library for rust.” <https://faer.veganb.tw/>.

Accessed: 2026-03-28.



J. Kitchin, “riopt: A memory-safe interior point optimizer written in rust, inspired by ipopt.”

<https://github.com/jkitchin/riopt>.

Accessed: 2026-04-11.